

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Expert System Development Project

COURSE NAME: ARTIFICIAL INTELLIGENCE COURSE CODE: MLA0101
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO 5: Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A

Expert System Development Project

Code of Conduct:

I Mangali Veera Naga Sai (192425253) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M Veera Naga Sai

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	
10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

Tool: SWI-Prolog

Usage of Prolog: Students shall use **Prolog** to develop a rule-based expert system by representing domain knowledge as **facts and production rules**, and implementing reasoning through **forward and backward chaining**. The project should demonstrate Prolog's **declarative/non-procedural nature**, logical inference, rule matching, and automated conclusion generation.

S.No.	Scenario-Based Expert System Development Question
1	Medical Diagnosis Expert System: Develop a rule-based expert system that identifies possible diseases based on symptoms, patient observations, and basic clinical information. Represent domain knowledge using production rules and implement both forward and backward chaining to derive conclusions.
2	Crop Disease Advisory System: Develop an expert system that identifies possible crop diseases based on leaf symptoms, soil conditions, weather conditions, and visible plant characteristics. Construct suitable production rules and demonstrate diagnosis using forward and backward chaining.
3	Automobile Fault Diagnosis System: Design an expert system that identifies probable vehicle faults based on symptoms such as engine noise, starting problems, warning indicators, abnormal vibration, and reduced mileage. Implement the knowledge base using production rules and compare forward and backward reasoning.
4	Student Academic Advisory System: Develop an expert system that recommends suitable academic interventions based on attendance, internal marks, assignment performance, learning difficulties, and previous academic performance. Use production rules and demonstrate both forward and backward chaining.
5	Cybersecurity Threat Diagnosis System: Construct a rule-based expert system that identifies possible cybersecurity threats from events such as repeated login failures, unusual network traffic, suspicious file activity, and unauthorized access attempts. Implement production rules and demonstrate how forward and backward chaining arrive at a threat classification.

Report Format:

Stage	Student Activity	Expected Output
1. Domain Analysis	Identify the problem domain, entities, facts, symptoms/conditions, and possible conclusions.	Problem definition and domain model
2. Knowledge Acquisition	Collect domain knowledge from reliable sources or domain experts.	Knowledge specification
3. Knowledge Representation	Convert domain knowledge into IF–THEN production rules.	Knowledge base
4. Inference Design	Implement forward chaining and backward chaining.	Inference engine
5. Paradigm Analysis	Distinguish how the solution can be expressed using procedural and non-procedural approaches.	Paradigm comparison
6. System Development	Integrate the knowledge base, inference engine, user interface, and explanation facility.	Working expert system
7. Testing	Test the system using multiple facts and scenarios.	Test cases and outputs
8. Evaluation	Analyze correctness, coverage, consistency, and reasoning performance.	Evaluation results
9. Documentation	Prepare technical documentation and GitHub repository.	Project report + GitHub
10. Demonstration	Demonstrate the system and explain the reasoning process.	Project presentation/viva

CROP DISEASE ADVISORY SYSTEM

Project Title:

Crop Disease Advisory System Using Expert System, Forward Chaining and Backward Chaining

Objective:

To develop an expert system that identifies possible crop diseases using leaf symptoms, soil conditions, weather conditions, and visible plant characteristics. The system uses IF–THEN production rules and demonstrates diagnosis using forward and backward chaining.

1. Domain Analysis

Stage	Student Activity	Expected Output
1.Domain Analysis	Identify the problem domain, entities, facts, symptoms/conditions, and possible conclusions.	Problem definition and domain model

Problem Definition

Farmers may find it difficult to identify crop diseases because different diseases can produce similar symptoms. The proposed system acts as an **expert advisor** by analyzing information about the crop and suggesting a possible disease.

Entities

- Crop
- Leaf
- Soil
- Weather
- Disease
- Symptoms
- Advisory

Input Facts

The system can consider:

Leaf symptoms

- Yellow leaves
- Brown spots
- White powder
- Leaf curling
- Leaf holes

2. Knowledge Acquisition:

Stage	Student Activity	Expected Output
Knowledge Acquisition	Collect domain knowledge from reliable sources or domain experts.	Knowledge specification

Knowledge Specification

Knowledge is collected by studying common crop diseases, their symptoms, environmental conditions, and suitable agricultural practices.

For example:

Disease	Important Symptoms	Conditions
Leaf Spot	Brown spots on leaves	High humidity
Powdery Mildew	White powder on leaves	Warm and humid conditions
Root Rot	Wilting, rotten roots	Excess soil moisture
Bacterial Blight	Water-soaked/brown leaf lesions	Rainy and humid weather
Mosaic Disease	Mosaic pattern, leaf curling	Virus/vector-related
Early Blight	Brown circular spots	Warm, humid conditions

Note: These rules are simplified educational rules for an expert-system project, not a substitute for professional agricultural diagnosis.

3. Knowledge Representation:

Stage	Student Activity	Expected Output
Knowledge Representation	Convert domain knowledge into IF–THEN production rules.	Knowledge base

4. Inference Design:

Stage	Student Activity	Expected Output
4. Inference Design	Implement forward chaining and backward chaining.	Inference engine

A. Forward Chaining

Forward chaining starts with **known facts** and applies rules until a conclusion is obtained.

Example

Given facts:

brown_spots = True

high_humidity = True

Rule:

IF brown_spots AND high_humidity

THEN Leaf_Spot

Reasoning

Fact 1 → Brown spots

Fact 2 → High humidity

↓

Rule 1

↓

Leaf Spot detected

Therefore:

Diagnosis = Leaf Spot

B. Backward Chaining

Backward chaining starts with a **possible conclusion** and works backward to check whether the required conditions are true.

Example

Suppose the system wants to verify:

Is the disease Leaf Spot?

It checks Rule 1:

IF brown_spots AND high_humidity

THEN Leaf_Spot

Now it checks:

brown_spots = True ✓

high_humidity = True ✓

Therefore:

Leaf_Spot = True

Backward reasoning

Goal: Leaf Spot

↓

Need: Brown spots

↓
 Need: High humidity
 ↓
 Both facts are available
 ↓
 Leaf Spot confirmed

5. Paradigm Analysis:

Stage	Student Activity	Expected Output
Paradigm Analysis	Distinguish how the solution can be expressed using procedural and non-procedural approaches.	Paradigm comparison

Procedural Approach

In a procedural approach, the programmer explicitly specifies **how** the diagnosis should be performed.

Example:

1. Read leaf symptoms.
2. Check soil moisture.
3. Check humidity.
4. Compare symptoms.
5. Display disease.

The program controls the sequence of operations.

Non-Procedural Approach

In the expert-system approach, we mainly specify **what is known** through rules.

Example:

IF brown_spots AND high_humidity
 THEN Leaf_Spot

The inference engine decides which rules should be applied.

Comparison

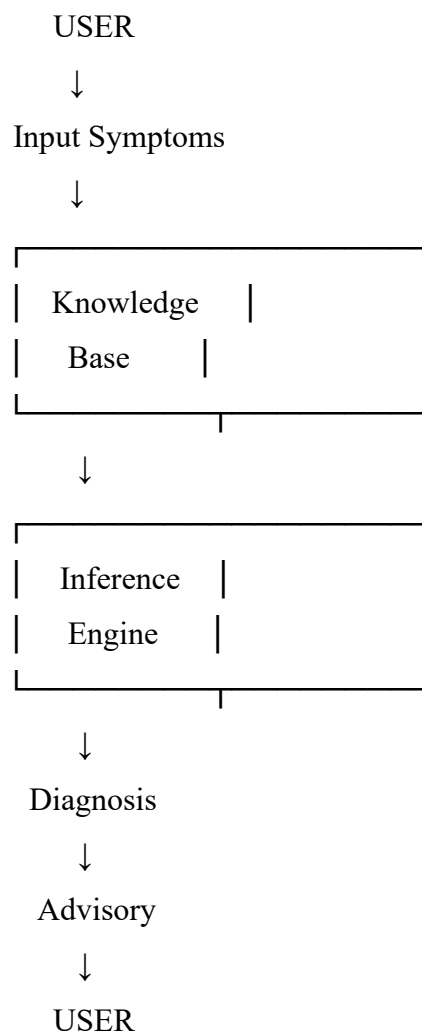
Procedural	Non-Procedural
Specifies how to solve	Specifies knowledge/rules
Uses explicit program flow	Uses inference engine

Procedural	Non-Procedural
Logic is embedded in code	Logic is stored in knowledge base
Less flexible for adding rules	Easier to add new rules
Example: traditional program	Example: expert system

6. System Development:

Stage	Student Activity	Expected Output
System Development	Integrate the knowledge base, inference engine, user interface, and explanation facility.	Working expert system

System Architecture



Main Components

1. User Interface

Accepts:

- Leaf symptoms
- Soil conditions
- Weather conditions
- Plant characteristics

2. Knowledge Base

Stores IF–THEN rules.

3. Inference Engine

Uses:

- Forward chaining
- Backward chaining

4. Explanation Facility

Explains why a particular disease was suggested.

Example:

Disease: Leaf Spot

Reason:

Brown spots were observed on the leaf
and humidity is high.

7. Testing:

Stage

Student Activity

Expected Output

7. Testing Test the system using multiple facts and scenarios. Test cases and outputs

Test Case 1 – Leaf Spot

Input

Brown spots: Yes

Humidity: High

Output

Possible Disease: Leaf Spot

Advisory: Remove affected leaves and improve air circulation.

Test Case 2 – Root Rot

Input

Wilting: Yes

Soil moisture: High

Poor drainage: Yes

Output

Possible Disease: Root Rot

Advisory: Reduce excess watering and improve soil drainage.

Test Case 3 – Powdery Mildew

Input

White powder on leaf: Yes

Temperature: Warm

Humidity: High

Output

Possible Disease: Powdery Mildew

Advisory: Improve ventilation and avoid excessive humidity.

Test Case 4 – Mosaic Disease

Input

Mosaic pattern: Yes

Leaf curling: Yes

Stunted growth: Yes

Output

Possible Disease: Mosaic Disease

Advisory: Inspect plants for viral/vector symptoms and isolate affected plants.

Test Case 5 – No Matching Disease

Input

Brown spots: No

White powder: No

Wilting: No

Mosaic pattern: No

Output

Disease: No matching disease found.

Recommendation: Consult an agricultural expert for further diagnosis.

8. Evaluation

Stage	Student Activity	Expected Output
8. Evaluation	Analyze correctness, coverage, consistency, and reasoning performance.	Evaluation results

Evaluation Criteria

1. Correctness

Check whether the system produces the expected disease for known test cases.

2. Coverage

Check whether the knowledge base covers different:

- Leaf symptoms
- Soil conditions
- Weather conditions
- Plant characteristics

3. Consistency

Rules should not produce contradictory conclusions for the same set of facts.

4. Reasoning Performance

Measure how quickly the system reaches a conclusion.

Sample Evaluation

Test	Expected	System Result
Brown spots + high humidity	Leaf Spot	Correct
Wilting + wet soil + poor drainage	Root Rot	Correct
White powder + warm + humid	Powdery Mildew	Correct
Mosaic + curling + stunted growth	Mosaic Disease	Correct
No symptoms	No diagnosis	Correct

9. Documentation

Stage	Student Activity	Expected Output
9. Documentation	Prepare technical documentation and repository.	GitHub Project report + GitHub

Project Documentation

The report should contain:

1. Introduction
2. Problem Definition
3. Objectives
4. Domain Analysis
5. Knowledge Acquisition
6. Knowledge Representation
7. Production Rules
8. Forward Chaining
9. Backward Chaining
10. System Architecture
11. Implementation
12. Test Cases
13. Results
14. Evaluation
15. Conclusion
16. Future Scope
17. References

Suggested GitHub Structure

Crop-Disease-Advisory-System/

```
|  
|— README.md  
|— crop_disease_system.py  
|— rules.txt  
|— test_cases.txt  
|— screenshots/  
|   |— input.png  
|   |— output.png  
|— report.pdf
```

10. Demonstration

Stage	Student Activity	Expected Output
10. Demonstration	Demonstrate the system and explain the reasoning process.	Project presentation/viva

Demonstration Example

Step 1 – Enter Facts

Leaf has brown spots: Yes

Humidity: High

Soil moisture: Normal

Plant wilting: No

Step 2 – Forward Chaining

The system checks the rules:

Brown spots = True

High humidity = True

↓

Rule 1 satisfied

↓

Leaf Spot

Step 3 – Display Result

CROP DISEASE ADVISORY SYSTEM

Disease: Leaf Spot

Reason:

1. Brown spots detected on leaf.
2. Humidity is high.
3. Rule 1 was satisfied.

Advisory:

Remove affected leaves and improve
air circulation.

Step 4 – Backward Chaining Demonstration

The system starts with:

Goal: Is this Leaf Spot?

Then checks:

Leaf Spot



Brown spots?



YES ✓

High humidity?



YES ✓

Therefore:

Leaf Spot confirmed.

Overall System Flow

START



Collect User Information



Leaf Symptoms

Soil Conditions

Weather Conditions

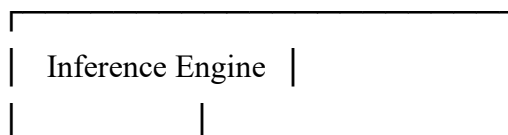
Plant Characteristics

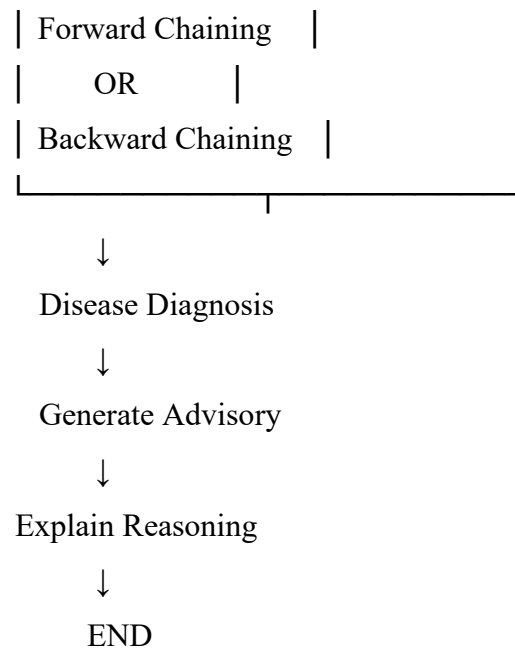


Store Facts



Apply Production Rules





Final Conclusion

The **Crop Disease Advisory System** is a rule-based expert system that uses agricultural knowledge represented as **IF–THEN production rules**. It accepts leaf symptoms, soil conditions, weather conditions, and plant characteristics as facts. The **forward-chaining mechanism** starts from observed symptoms and derives a disease, while **backward chaining** starts from a suspected disease and verifies whether its required conditions are satisfied. The system can help demonstrate how expert knowledge and AI inference techniques can be applied to agricultural disease diagnosis.

EVALUATION RUBRICS:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, facts, conditions, goals, and expected conclusions.	7–8% – Good understanding of the domain with minor omissions.	5–6% – Basic domain understanding with several missing elements.	0–4% – Poor or incorrect understanding of the problem domain.

2. Knowledge Acquisition and Representation	10%	9–10% – Acquires relevant domain knowledge and represents it accurately using well-structured Prolog facts and rules.	7–8% – Knowledge is appropriately represented with minor gaps.	5–6% – Basic knowledge representation with missing or unclear facts/rules.	0–4% – Inadequate or inappropriate knowledge representation.
3. Production Rule / Knowledge Base Design	15%	13–15% – Develops a comprehensive, consistent, and logically structured production-rule knowledge base with appropriate facts and rules.	10–12% – Well-designed knowledge base with minor inconsistencies or omissions.	7–9% – Basic rule base developed but several rules are incomplete or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.
4. Prolog Implementation	15%	13–15% – Implements a fully functional expert system using Prolog facts, predicates, rules, variables, unification, and backtracking effectively.	10–12% – Functional Prolog implementation with minor technical issues.	7–9% – Basic implementation works with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.
5. Forward Chaining	10%	9–10% – Correctly implements and demonstrates forward chaining from initial facts through applicable rules to final conclusions.	7–8% – Forward chaining works correctly with minor limitations.	5–6% – Basic forward reasoning demonstrated but with incomplete rule processing.	0–4% – Forward chaining is missing or incorrectly implemented.

6. Backward Chaining	10%	9–10% – Effectively demonstrates goal-driven backward chaining using Prolog's inference mechanism and clearly traces the reasoning process.	7–8% – Backward chaining works correctly with minor gaps.	5–6% – Basic backward reasoning demonstrated with limited explanation.	0–4% – Backward chaining is missing or incorrectly implemented.
7. Procedural and Non-Procedural Paradigm Analysis	10%	9–10% – Clearly distinguishes procedural and declarative/non-procedural approaches and demonstrates their application in the Prolog system.	7–8% – Provides a good distinction with minor conceptual gaps.	5–6% – Basic distinction is explained but lacks practical demonstration.	0–4% – Paradigms are misunderstood or not addressed.
8. Inference, Unification and Reasoning Accuracy	10%	9–10% – Produces accurate conclusions using unification, backtracking, and logical inference across diverse test cases.	7–8% – Reasoning is mostly accurate with minor errors.	5–6% – Basic inference works but produces occasional incorrect results.	0–4% – Inference is unreliable or incorrect.
9. Testing, Results and System Demonstration	5%	5% – Provides comprehensive test cases, accurate outputs, reasoning traces, and a convincing live demonstration.	4% – Good testing and demonstration with minor gaps.	2–3% – Limited test cases and basic demonstration.	0–1% – Testing or demonstration is missing/inadequate.
10. Documentation, GitHub and Technical Presentation	5%	5% – Complete documentation, well-organized GitHub repository, clear code, references, and professional presentation.	4% – Good documentation and repository with minor issues.	2–3% – Basic documentation/repository with several omissions.	0–1% – Poor documentation, missing repository, or inadequate presentation.

Total	100%				
-------	------	--	--	--	--